

PAI 4 - DevSecOps

Seguridad en Sistemas Informaticos y en Internet Security Team 9 Fernando Cobos Garcia David Escudero
Aldana Jesus Oria Arenas 27/04/2026

Resumen ejecutivo

Este proyecto implementa una pipeline DevSecOps completa sobre una aplicacion web minima en Flask. El objetivo no ha sido construir una aplicacion compleja, sino una entrega clara, reproducible y trazable que permita demostrar seguridad integrada en todo el ciclo de vida del desarrollo.

La solucion final incluye:

- una pipeline GitLab CI/CD con las fases `sca -> sast -> iac -> test -> build -> deploy -> dast -> vulnerability-management`
- una aplicacion de prueba con autenticacion, ruta protegida, control de autorizacion, formulario validado y logica de negocio en servidor
- integracion real de herramientas SCA, SAST, IaC y DAST
- un conjunto nuevo de pruebas de seguridad estables y orientadas a controles reales
- evidencias verificables almacenadas en `reports/`

La referencia del proyecto anterior en `Fran/` se utilizo solo para detectar problemas a evitar. No se reutilizo su implementacion porque contenia tests no fiables y configuraciones poco seguras.

Parte I. Informe tecnico

1. Alcance y arquitectura de la solucion

La solucion se ha implementado integramente en la carpeta `Nuevo/` y se apoya en una aplicacion web Flask con almacenamiento SQLite, empaquetado Docker y automatizacion mediante GitLab CI.

Elemento	Implementacion
Aplicacion web	Flask
Almacenamiento	SQLite
Autenticacion	Sesion de Flask
Autorizacion	Decoradores por rol
Contenedorizacion	Docker
Pipeline	GitLab CI/CD
SCA	pip-audit
SAST	Semgrep
IaC	Trivy
DAST	OWASP ZAP
Gestion de vulnerabilidades	DefectDojo via API

2. Aplicacion utilizada para la demostracion

La aplicacion se ha mantenido deliberadamente pequena para poder aislar mejor los controles de seguridad y los hallazgos de cada fase.

Requisito del proyecto	Implementacion
Login	`/login`
Ruta protegida	`/dashboard`
Control de autorizacion	`/admin/audit`
Formulario con entrada de usuario	`/feedback`
Integridad de negocio	`/checkout`
Vulnerabilidad intencional	`/legacy/search`
Verificacion de despliegue	`/health`

Los hallazgos intencionales introducidos para demostrar la pipeline son:

- una dependencia vulnerable en `security/sca/requirements-sca.txt`
- una salida insegura con `Markup(query)` en `app/main.py`
- un Dockerfile sin usuario no privilegiado
- ausencia de ciertas cabeceras y protecciones para que ZAP detecte findings reales

3. Pipeline CI/CD seleccionada

La pipeline se implementa en `.gitlab-ci.yml` y sigue exactamente el orden solicitado por el enunciado:

```
sca -> sast -> iac -> test -> build -> deploy -> dast -> vulnerability-management
```

Cada fase ejecuta un script dedicado y genera artefactos persistentes en `reports/`.

	Objetivo	Script	Evidencia principal
	detectar vulnerabilidades en dependencias	`scripts/run_sca.sh`	`reports/sca/pip-audit-report.json`
	detectar patrones inseguros en codigo	`scripts/run_sast.sh`	`reports/sast/semgrep.json`
	detectar malas practicas del Dockerfile	`scripts/run_iac.sh`	`reports/iac/trivy-iac-report.json`
	verificar controles de seguridad reales	`scripts/run_tests.sh`	`reports/tests/pytest-report.json`
	construir la imagen Docker	`scripts/run_build.sh`	`reports/build/docker-build.log`
	levantar la aplicacion para validacion	`scripts/run_deploy.sh`	`reports/deploy/healthcheck.json`
	escanear la aplicacion en ejecucion	`scripts/run_dast.sh`	`reports/dast/zap-report.json`
vulnerability management	agregar resultados en gestor central	`scripts/run_defectdojo.sh`	`reports/vulnerability-management/REAL`

4. Herramientas seleccionadas e integracion

4.1. SCA - pip-audit

Se usa `pip-audit` para analizar `security/sca/requirements-sca.txt`. Ese manifiesto amplía las dependencias de runtime e introduce `urllib3==1.25.8` para asegurar que el escaneo produce evidencia real.

Evidencias:

- `reports/sca/pip-audit.txt`
- `reports/sca/pip-audit-report.json`
- `reports/sca/pip-audit.log`

Resultado observado:

- detección de múltiples advisories sobre `urllib3==1.25.8`

4.2. SAST - Semgrep

Se usa Semgrep con una regla local en `security/sast/semgrep-rules.yml`. La regla detecta el uso de `Markup(...)` sobre entrada controlada por el usuario, lo que reintroduce riesgo de XSS.

Evidencias:

- `reports/sast/semgrep.txt`
- `reports/sast/semgrep.json`
- `reports/sast/semgrep.log`

Resultado observado:

- un hallazgo en `app/main.py` relacionado con la ruta `/legacy/search`

4.3. IaC - Trivy

Se usa Trivy para analizar el Dockerfile en `docker/Dockerfile`.

Evidencias:

- `reports/iac/trivy-iac.txt`
- `reports/iac/trivy-iac-report.json`
- `reports/iac/trivy.log`

Resultados observados:

- `DS-0002` por ausencia de usuario no root
- `DS-0026` por ausencia de `HEALTHCHECK`

4.4. DAST - OWASP ZAP

Se usa `zap-full-scan.py` contra la aplicación desplegada en contenedor.

Evidencias:

- `reports/dast/zap-report.html`
- `reports/dast/zap-report.json`
- `reports/dast/zap-report.xml`
- `reports/dast/zap.log`

Resultados observados:

- cabeceras de seguridad ausentes
- ausencia de anti-CSRF en el login
- XSS reflejado y DOM-based sobre `/legacy/search`

5. Pruebas de seguridad implementadas

Los tests del proyecto anterior se descartaron por no ser fiables. En esta entrega se ha creado una suite nueva en `tests/security/`, orientada a controles de seguridad reales y sin dependencias fragiles de mensajes o plantillas.

Control verificado	Test
autenticacion obligatoria	`test_dashboard_requires_login`
login correcto	`test_successful_login_sets_session_and_redirects_to_dashboard`
autorizacion por rol	`test_member_cannot_access_admin_route` y `test_admin_can_access_admin_route`
validacion de entrada	`test_feedback_rejects_overlong_input` y `test_checkout_rejects_invalid_quantities`
prevencion XSS en salida segura	`test_feedback_output_is_escaped`
no exposicion de datos sensibles	`test_profile_omits_password_material`
integridad de negocio	`test_checkout_total_is_calculated_server_side`

Evidencias:

- `reports/tests/pytest.log`
- `reports/tests/pytest-report.json`
- `reports/tests/pytest-junit.xml`

Resultado observado:

- `9 passed`

6. Gestion de vulnerabilidades

Se ha preparado integracion con DefectDojo mediante API REST contra `/api/v2/reimport-scan/`. La fase consume directamente los reportes generados por SCA, SAST, IaC y DAST.

Evidencias:

- `scripts/run\_defectdojo.sh`
- `reports/vulnerability-management/attempts.tsv`
- `reports/vulnerability-management/README.md`

Estado actual:

- la integracion esta implementada
- el flujo de importacion esta preparado
- no se ha validado una importacion real por no disponer de `DEFECTDOJO\_URL` y `DEFECTDOJO\_API\_TOKEN`

Esta limitacion no invalida la entrega porque la herramienta de gestion ha sido seleccionada, integrada y documentada de forma trazable.

Parte II. Manual de despliegue y uso

1. Requisitos previos

Para ejecutar la solucion localmente se necesita:

- Python 3.11 o superior
- Docker Desktop en ejecucion
- acceso a Internet para descargar dependencias e imagenes

2. Puesta en marcha rapida

2.1. Tests de seguridad

```
python -m venv .venv
.\.venv\Scripts\python -m pip install --upgrade pip
.\.venv\Scripts\python -m pip install -r requirements.txt
$env:PYTHONPATH='.'
.\.venv\Scripts\python -m pytest tests\security
```

Resultado esperado:

- `9 passed`

2.2. Arranque de la aplicacion

```
$env:PAI4_ADMIN_PASSWORD="AdminPassTemporal!123"
$env:PAI4_MEMBER_PASSWORD="MemberPassTemporal!123"
docker compose -f docker\docker-compose.yml up --build
```

La aplicacion queda disponible en:

- `http://localhost:5000/`
- `http://localhost:5000/login`
- `http://localhost:5000/health`

2.3. Reproduccion de fases de seguridad

La pipeline utiliza estos scripts:

- `scripts/run\_sca.sh`
- `scripts/run\_sast.sh`
- `scripts/run\_iac.sh`
- `scripts/run\_tests.sh`
- `scripts/run\_build.sh`
- `scripts/run\_deploy.sh`
- `scripts/run\_dast.sh`
- `scripts/run\_defectdojo.sh`

3. Verificaciones minimas recomendadas

Comprobacion	Resultado esperado
`/dashboard` sin login	redireccion a login
test suite	`9 passed`
SCA	findings reales
SAST	1 hallazgo sobre `Markup(query)`
IaC	findings `DS-0002` y `DS-0026`
`/health`	`{"status":"ok"}`
ZAP	reportes con findings reales

4. Ubicacion de evidencias

Categoria	Ruta
SCA	`reports/sca/`
SAST	`reports/sast/`
IaC	`reports/iac/`
Tests	`reports/tests/`
Build	`reports/build/`
Deploy	`reports/deploy/`
DAST	`reports/dast/`
Vulnerability management	`reports/vulnerability-management/`

Parte III. Grado de completitud

1. Cumplimiento de objetivos

	Estado	Evidencia
pipeline CI/CD	Completo	`gitlab-ci.yml`

	Estado	Evidencia
al menos tres herramientas de seguridad	Completo	`pip-audit`, `Semgrep`, `Trivy`, `OWASP ZAP`
herramientas en el ciclo de vida	Completo	scripts y artefactos por fase
pruebas para detectar vulnerabilidades	Completo	`tests/security/` y `reports/tests/`
e integrar herramienta de gestion	Completo con limitacion documentada	`scripts/run_defectdojo.sh` y `reports/vulnerability`

2. Cumplimiento de normas del entregable

Requisito formal	Estado	Observacion
Codigo fuente y scripts	Completo	incluido en `codigo/` dentro del ZIP
Tests, logs y evidencias	Completo	incluidos en `reports/`
Informe tecnico en PDF	Completo	`Informe-PAI4.pdf`
Manual de despliegue y uso	Completo	incluido en el PDF
Grado de completitud trazable	Completo	incluido en el PDF
Maximo 10 paginas	Completo	PDF final inferior a 10 paginas

3. Conclusiones finales

La entrega satisface los objetivos tecnicos del PAI-4 y presenta una cadena DevSecOps coherente, reproducible y defendible. La solucion prioriza claridad, evidencia y trazabilidad por encima de complejidad innecesaria, lo que facilita tanto la correccion como la demostracion practica ante el profesor.